Case Study: Climate Hazard Trend Analyzer

Full-Stack Developer role at Dygnify Ventures Pvt Ltd

Before You Start

- **Time:** This is designed for about 8–10 hours of focused work. The submission deadline (IST) is in the LinkedIn message or job post that shared this case study.
- **How to submit:** Use the submission form link shared with this case study. Submit once. To correct something, resubmit with the same email address; only your latest submission is kept.
- **Scope over polish:** A working, well-reasoned heatwave analysis beats a half-finished multi-hazard app. State any assumptions in your README.
- **AI tools:** You may use them. Tell us which ones you used and how in your README and the submission form. In the interview you will walk us through your code and make changes to it live.
- **Confidentiality:** Please do not publish or share this brief. Your code repository can be public.

Problem Statement

Build a web application that analyzes historical weather data and visualizes trends in climate hazards, such as heatwaves, droughts, or heavy rainfall events, for a selected region over time.

Key Features

Frontend (React)

Allow users to:

- Select a geographic region (city, state, or a custom lat-lon box).
- Use Heatwave as the hazard type, selected by default.
- Choose a time range (e.g. 1990–2020).

Display:

- Time-series plots of hazard frequency/intensity.
- Interactive visualizations (e.g. charts, maps).
- Summary insights such as “Heatwaves have increased by 40% in this region since 1990.”

Backend (Python)

Ingest and cache historical weather data from public APIs such as:

- NOAA NCEI
- Copernicus Climate Data Store
- Open-Meteo API
- Or any other source

Define logic to detect climate hazards:

- **Example (Heatwave):** 3+ consecutive days with maximum temperature above the 95th percentile for that region.
- Analyze frequency and duration trends using rolling averages or regression.
- Store this data in a NoSQL database.
- On an API call, read this data from the database and return JSON summaries and trend plots.

Evaluation Dimensions

Dimension	What we look for	Weight
Data Handling	Working with real-world noisy, inconsistent data: gaps, outliers, units, caching	20%
Climate Logic	Ability to define and apply domain-specific rules (e.g. a percentile-based heatwave definition)	20%
Backend Skills	Efficient data processing, clean API design, safe external API access	20%
Frontend Skills	Clear, interactive visualization using Chart.js, D3.js, or similar	15%
Critical Thinking	Deriving sound, honest insight from climate indicators	10%
Security + Performance	API caching, rate-limiting, and safe handling of user inputs	15%

Bonus Challenges

These are optional and add up to 10 extra points. Attempt them only after the core features work.

- Use a map UI (e.g. Leaflet or Mapbox) for region selection.
- Provide PDF/CSV export of insights.
- Apply trend detection techniques such as linear regression or the Mann-Kendall test.

What to Submit

- ☐ Public GitHub repository link
- ☐ Deployed app URL (any free tier works, e.g. Vercel, Netlify, Firebase Hosting, Render, Cloud Run)
- ☐ README covering: setup steps, architecture, data source(s), hazard logic, assumptions, and what you would improve with more time
- ☐ AI tools used and how (in the README)
- ☐ Optional: a 3–5 minute demo video (Loom or Google Drive link)

What Happens Next

- We review every submission received by the deadline. Late submissions may not be reviewed.
- Selection has two steps: this case study, then an interview with shortlisted candidates based on their submitted solution.
- We cannot answer any further questions about the case study or the solution itself. Use your judgment and note your assumptions.